Amazon DynamoDB

1. Fully Managed and Highly Available
DynamoDB is a fully managed database service, meaning AWS takes care of the database infrastructure, servers, hardware, maintenance, and patching. We don't need to manage database servers manually. DynamoDB also provides high availability by storing data redundantly across multiple Availability Zones.
Simple ga gurthupettuko:
Fully Managed = AWS manages infrastructure
Highly Available = Data available across multiple AZs

---

2. NoSQL Database
DynamoDB is a **NoSQL database**, not a traditional relational database like MySQL or PostgreSQL. It uses a key-value and document-based data model, so data can be stored in a flexible structure without requiring the same fixed columns for every item.
Simple ga gurthupettuko:
DynamoDB = NoSQL
MySQL/PostgreSQL = Relational SQL

---

3. Scales to Massive Workloads
DynamoDB is designed to handle very large and distributed workloads. As the number of users and requests increases, DynamoDB can support large-scale applications without us having to manually manage database servers.
Simple ga gurthupettuko:
DynamoDB = Massive workload ni handle cheyyagaladu

---

4. Millions of Requests and Huge Data
DynamoDB is designed to handle very high request rates and very large amounts of data. It can support applications with millions of requests per second and very large datasets, making it suitable for applications that need massive scale.
Example:
An e-commerce application may have millions of users accessing product, order, and customer data at the same time.
Simple ga gurthupettuko:
DynamoDB = Millions of requests + Huge amount of data

---

5. Fast and Low-Latency Performance
DynamoDB is designed to provide fast and consistent performance with low latency. Low latency ante user request chesina tarvata database response very quickly ravatam. This is useful for applications such as gaming, shopping, and real-time applications where fast data access is important.
Simple ga gurthupettuko:
Low Latency = Fast response

---

6. Integrated with IAM
DynamoDB integrates with **AWS IAM** for security and authorization. IAM policies can control which users, roles, or applications can access DynamoDB and what actions they are allowed to perform, such as reading or writing data.
Example:
User / Application
↓
IAM
↓
DynamoDB
↓
Read / Write
Simple ga gurthupettuko:
IAM + DynamoDB = Access Control and Security

---

7. DynamoDB Streams
DynamoDB Streams records changes made to items in a DynamoDB table, such as when an item is created, updated, or deleted. These changes can be used with services like AWS Lambda to build event-driven applications.
Example:
New Order
↓
DynamoDB
↓
DynamoDB Streams

↓
Lambda
↓
Send Notification
Simple ga gurthupettuko:
DynamoDB Streams = Capture database changes

---

8. Low Cost and Auto Scaling
DynamoDB provides different capacity and pricing options, and it supports automatic scaling so the database can adjust capacity based on application traffic when configured appropriately. This helps us handle changing workloads without manually increasing or decreasing database capacity.
Example:
Low Traffic
↓
Lower Capacity

High Traffic
↓
Higher Capacity
Simple ga gurthupettuko:
Auto Scaling = Traffic batti capacity adjust cheyyadam

⭐ **Final Memory**
Amazon DynamoDB
│
├— Fully Managed
├— Highly Available
├— NoSQL
├— Massive Scale
├— Low Latency
├— IAM Security
├— DynamoDB Streams
└— Auto Scaling
One-line interview memory:
DynamoDB is a fully managed, highly available NoSQL database that provides low-latency performance, massive scalability, IAM-based security, Streams, and automatic scaling.

Amazon DynamoDB — Important Interview Questions & Answers
1. What is a Table?
Answer:
A Table is a place where DynamoDB stores data.
It contains multiple items (records) and attributes (fields).
For example, a Users table can store user information.
Simple ga gurthupettuko:
Table = Collection of data

---

2. What is an Item?
Answer:
An Item is one record inside a DynamoDB table.
It contains different attributes related to one object.
For example, one user record is one Item.
Simple ga gurthupettuko:
Item = One record

---

3. What is an Attribute?
Answer:
An Attribute is a piece of information inside an Item.
It is similar to a column value in a traditional database.
For example, Name, Age, and Email are attributes.

Simple ga gurthupettuko:
Attribute = Field / Data value

---

4. What is a Primary Key?
Answer:
A Primary Key uniquely identifies each Item in a DynamoDB table.
DynamoDB uses the Primary Key to find an Item quickly.
It can be a Partition Key or Partition Key + Sort Key.
Simple ga gurthupettuko:
Primary Key = Uniquely identifies Item

---

5. What is a Partition Key?
Answer:
A Partition Key is used to determine where an Item is stored.
DynamoDB uses its value to distribute data across partitions.
Its value must be unique if it is used alone as the Primary Key.
Simple ga gurthupettuko:
Partition Key = Distribute and find data

---

6. What is a Sort Key?
Answer:
A Sort Key is used together with a Partition Key.
It helps organize multiple Items having the same Partition Key.
It also allows us to query Items based on the Sort Key.
Simple ga gurthupettuko:
Sort Key = Organize Items within Partition Key

---

7. What is a Composite Primary Key?
Answer:
A Composite Primary Key contains two parts: Partition Key and Sort Key.
The Partition Key groups the Items, and Sort Key differentiates them.
It is useful when multiple Items belong to the same Partition Key.
Simple ga gurthupettuko:
Composite Key = Partition Key + Sort Key

---

8. What is Query?
Answer:
Query is used to find Items using a specific Partition Key.
It can also use conditions on the Sort Key.
Query is normally faster and more efficient than Scan.
Simple ga gurthupettuko:
Query = Find specific data

---

9. What is Scan?
Answer:
Scan checks every Item in a DynamoDB table or index.
It can be used when we don't know the Partition Key.
But Scan can consume more capacity and be slower for large tables.
Simple ga gurthupettuko:
Scan = Check all Items

---

10. What is the difference between Query and Scan?
Answer:
Query finds Items using the Partition Key, while Scan checks all Items.
Query is generally faster and uses capacity more efficiently.
Scan should be avoided for frequent large-table searches.
Simple ga gurthupettuko:
Query = Specific data | Scan = All data

---

11. What is Global Secondary Index (GSI)?

Answer:
GSI is an additional index that allows us to query data using different keys.
It can have a different Partition Key and Sort Key from the main table.
A GSI can be created even after the table is created.
Simple ga gurthupettuko:
GSI = Search data using another key

---

12. What is Local Secondary Index (LSI)?
Answer:
LSI is an additional index that uses the same Partition Key as the main table.
It can have a different Sort Key.
LSI must be created when the table is created.
Simple ga gurthupettuko:
LSI = Same Partition Key + Different Sort Key

---

13. What is the difference between GSI and LSI?
Answer:
GSI can use a different Partition Key, while LSI uses the same Partition Key.
GSI can be created after table creation, but LSI must be created during table creation.
GSI and LSI are both used for alternative ways to query data.
Simple ga gurthupettuko:
GSI = Different PK | LSI = Same PK

---

14. What is Provisioned Capacity?
Answer:
Provisioned Capacity means we manually configure read and write capacity.
We specify how much RCU and WCU the table should have.
It is useful when traffic is predictable.
Simple ga gurthupettuko:
Provisioned = We choose capacity

---

15. What is On-Demand Capacity?
Answer:
On-Demand Capacity automatically handles read and write capacity based on traffic.
We don't need to specify RCU and WCU beforehand.
It is useful for unpredictable or changing workloads.
Simple ga gurthupettuko:
On-Demand = AWS manages capacity

---

16. What is Read Capacity Unit (RCU)?
Answer:
RCU represents the read capacity available for a DynamoDB table.
One RCU provides one strongly consistent read per second for up to 4 KB.
For eventually consistent reads, one RCU provides two reads per second for up to 4 KB.
Simple ga gurthupettuko:
RCU = Read capacity

---

17. What is Write Capacity Unit (WCU)?
Answer:
WCU represents the write capacity available for a DynamoDB table.
One WCU supports one write per second for an item up to 1 KB.
More data or more writes require more WCUs.
Simple ga gurthupettuko:
WCU = Write capacity

---

18. What is Eventually Consistent Read?
Answer:
Eventually Consistent Read may not immediately return the latest data.
The updated data becomes consistent after a short time.
It uses less read capacity than a strongly consistent read.

Simple ga gurthupettuko:
Eventually Consistent = Latest data may take little time

---

19. What is Strongly Consistent Read?
Answer:
Strongly Consistent Read returns the latest successful write before the read.
It gives more immediate consistency than eventually consistent reads.
It requires more read capacity.
Simple ga gurthupettuko:
Strongly Consistent = Latest data immediately

---

20. What are Conditional Writes?
Answer:
Conditional Writes perform a write only when a specified condition is true.
This helps prevent unwanted or incorrect updates.
For example, update an item only if the current value matches a condition.
Simple ga gurthupettuko:
Conditional Write = Write only if condition is true

---

21. What are Transactions?
Answer:
DynamoDB Transactions allow multiple operations to succeed or fail together.
They help maintain consistency across multiple Items or Tables.
If one required operation fails, the transaction can be rolled back.
Simple ga gurthupettuko:
Transaction = All operations together

---

22. What are DynamoDB Streams?
Answer:
DynamoDB Streams records changes made to Items in a table.
It can capture events such as Item creation, update, and deletion.
It is commonly used with AWS Lambda for event-driven applications.
Simple ga gurthupettuko:
Streams = Track table changes

---

23. What is TTL?
Answer:
TTL means Time to Live.
It automatically deletes Items after a specified expiration time.
It is useful for temporary data such as sessions or old records.
Simple ga gurthupettuko:
TTL = Automatically delete expired data

---

24. What is Point-in-Time Recovery?
Answer:
Point-in-Time Recovery (PITR) continuously protects DynamoDB table data.
It allows us to restore the table to a specific point in time.
It is useful for recovering from accidental changes or deletion.
Simple ga gurthupettuko:
PITR = Restore table to a specific time

---

25. What is On-Demand Backup?
Answer:
On-Demand Backup is a manual backup of a DynamoDB table.
We can create it whenever we need a backup copy.
It is useful before major changes or for long-term backup requirements.
Simple ga gurthupettuko:
On-Demand Backup = Manual backup

---

26. What is Restore?

Answer:
Restore means creating a DynamoDB table from a backup or recovery point.
We can restore data from On-Demand Backup or Point-in-Time Recovery.
It is useful when the original table data is lost or damaged.
Simple ga gurthupettuko:
Restore = Recover table from backup

---

27. What are Global Tables?
Answer:
Global Tables replicate DynamoDB data across multiple AWS Regions.
They are useful for globally distributed applications.
Users in different Regions can access nearby database replicas.
Simple ga gurthupettuko:
Global Tables = DynamoDB across multiple Regions

---

28. What is Encryption in DynamoDB?
Answer:
DynamoDB automatically encrypts data at rest.
We can use AWS-owned keys or AWS KMS keys depending on the requirement.
Encryption helps protect stored database data.
Simple ga gurthupettuko:
Encryption = Protect data at rest

---

29. What is IAM in DynamoDB?
Answer:
IAM controls who can access DynamoDB resources.
We can allow or deny actions such as read, write, update, or delete.
IAM policies are used to control these permissions.
Simple ga gurthupettuko:
IAM = Who can access DynamoDB

---

30. What is Fine-Grained Access?
Answer:
Fine-Grained Access allows more specific control over DynamoDB data access.
We can control access based on tables, items, or attributes using appropriate policies.
It helps give users only the permissions they actually need.
Simple ga gurthupettuko:
Fine-Grained Access = Detailed data-level permissions

---

31. What is DynamoDB Accelerator (DAX)?
Answer:
DAX is an in-memory caching service designed for DynamoDB.
It can reduce read latency by serving frequently accessed data from cache.
It is useful for applications with very high read traffic.
Simple ga gurthupettuko:
DAX = Cache for DynamoDB

---

32. What is the difference between DAX and DynamoDB?
Answer:
DynamoDB is the main database where data is stored.
DAX is a cache placed in front of DynamoDB to provide faster reads.
DAX does not replace DynamoDB as the main data store.
Simple ga gurthupettuko:
DynamoDB = Database | DAX = Cache

---

33. What is a Hot Partition?
Answer:
A Hot Partition happens when too much traffic goes to one partition.
This usually happens because of a poorly distributed Partition Key.
It can cause throttling and reduced performance.

Simple ga gurthupettuko:
Hot Partition = Too much traffic on one partition

---

34. What is Partition Key Design?
Answer:
Partition Key Design means choosing a key that distributes traffic and data evenly.
We should avoid keys where most requests use the same value.
A good Partition Key helps DynamoDB scale efficiently.
Simple ga gurthupettuko:
Good PK = Even data + traffic distribution

---

35. What is Throttling?
Answer:
Throttling happens when DynamoDB cannot handle the requested read or write traffic at that moment.
It can happen because of insufficient capacity or uneven traffic distribution.
DynamoDB returns throttling-related errors when requests are limited.
Simple ga gurthupettuko:
Throttling = Too much traffic for available capacity

---

36. What is ProvisionedThroughputExceededException?
Answer:
This error occurs when a request exceeds the available provisioned capacity.
It can happen because of high traffic or a hot partition.
We can handle it using retries with exponential backoff and proper capacity/key design.
Simple ga gurthupettuko:
Exception = Capacity exceeded

---

37. What is Auto Scaling?
Answer:
DynamoDB Auto Scaling automatically adjusts provisioned capacity based on workload.
It can increase or decrease RCU and WCU within configured limits.
It helps maintain performance while controlling capacity usage.
Simple ga gurthupettuko:
Auto Scaling = Automatically adjust capacity

---

38. How do you monitor DynamoDB using CloudWatch?
Answer:
CloudWatch provides metrics to monitor DynamoDB performance and usage.
We can check metrics such as consumed capacity, throttled requests, and latency.
It helps us identify performance or capacity problems.
Simple ga gurthupettuko:
CloudWatch = Monitor DynamoDB

---

39. How is DynamoDB used with Lambda?
Answer:
Lambda can read from and write data to DynamoDB.
DynamoDB Streams can also trigger Lambda when table data changes.
This is commonly used for event-driven applications.
Simple ga gurthupettuko:
DynamoDB + Lambda = Event-driven applications

---

40. How is DynamoDB used with API Gateway?
Answer:
API Gateway can receive API requests from users or applications.
Lambda can process the request and read or write data in DynamoDB.
This is a common serverless application architecture.
Simple ga gurthupettuko:
API Gateway → Lambda → DynamoDB

---

41. What is the difference between DynamoDB and RDS?

Answer:
DynamoDB is a NoSQL database, while RDS is used for relational databases.
DynamoDB uses tables with Items and Attributes, while RDS uses SQL tables and relationships.
DynamoDB is commonly used for massive scalable workloads with low latency.
Simple ga gurthupettuko:
DynamoDB = NoSQL | RDS = Relational SQL

---

42. What is the difference between DynamoDB and Aurora?
Answer:
DynamoDB is a NoSQL database designed for key-value and document workloads.
Aurora is a relational database compatible with MySQL and PostgreSQL.
Choose DynamoDB for NoSQL scalability and Aurora for relational SQL requirements.
Simple ga gurthupettuko:
DynamoDB = NoSQL | Aurora = SQL

---

43. What are common real-time DynamoDB troubleshooting scenarios?
Answer:
Common problems include throttling, hot partitions, high latency, and insufficient capacity.
First, we check CloudWatch metrics and identify the affected table or operation.
Then we check Partition Key design, capacity, traffic pattern, and application retries.
Example:
High Traffic
↓
Throttling
↓
CloudWatch
↓
Check Hot Partition / Capacity
↓
Fix Partition Key or Capacity
Simple ga gurthupettuko:
Problem → CloudWatch → Find Cause → Fix